



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Ingegneria

Corso di Laurea in Ingegneria Informatica

Resiliency Real-Time and Certification

AI Safety Critical Runtime Monitoring

Edoardo Sarri

7173337

Giugno 2026

Contents

1	Introduction	4
1.1	AI Tools Used	4
1.2	Terms	4
2	Overview	6
2.1	Literature Classification	6
2.1.1	Types of AI	6
2.1.2	Automation Levels	6
2.1.3	Safety Standards	7
2.1.4	Points of Application	7
2.2	AI-Based Safety-Critical Systems	8
2.2.1	Product	9
2.2.2	Runtime	10
2.2.3	Process	10
3	Runtime Monitoring	11
3.1	Architecture	11
3.2	Coverage	12
4	Implementation	15
4.1	Scenario	15
4.1.1	Components	16
4.1.2	Acceleration Function	16
4.2	Simulator	17
4.2.1	Kinematics Rules	17
4.2.2	State	18
4.2.3	Initial Conditions	18
4.3	Architecture	18
4.3.1	AI Module	18
4.3.2	Safety Monitor	19
4.3.3	Recovery Function	19
4.4	Implementation Details	19
4.4.1	Algorithm	19
4.4.2	Environment	19
4.4.3	Reward Function	20
4.4.4	Episode Termination	20
4.5	Results	20

List of Figures

2.1	AI-Based safety-critical system components [4].	8
3.1	RTA architecture Proposed in ASTM F3269-17 [3].	12
3.2	RTA architecture with Hierarchical Monitor [2].	13
3.3	Gap in the Monitor and the Recovery Function [3].	13
3.4	Voter architecture with multiple Monitors [3].	14
4.1	Activity Diagram	15
4.2	Sequence diagram.	16

1 Introduction

In recent years, an interesting challenge in the field of Artificial Intelligence (AI) and safety-critical systems has been the integration of AI models into systems that require predictable and safe behavior. The black-box nature of AI models implies a lack of structured standards and methodologies to build AI-based safety-critical systems.

In this project, I begin my study of these topics by examining one of the key papers [4] in this discipline.

This is the [repository](#) of the project.

1.1 AI Tools Used

During the development of this project, in order to improve the report, the development process, and the quality of the overall work, I used the following AI tools:

- **NotebookLM**

It was useful for performing an initial review of the reference paper [4]. [Here](#) is the link to the notebook; although the chat history is no longer available, you can see the main interactions, such as the concept map and report.

- **Copilot completions**

While writing this report (and the slides), Copilot completions generated some parts of the text for me.

- **Antigravity CLI**

The CLI version of Gemini was used to check the grammatical correctness of my English after the writing of the whole report. It also used to write the implementation of the interfaces after defining them.

1.2 Terms

In this short section, we clarify some of the terms used in the remainder of this work.

- **AI and Machine Learning**

This refers to the algorithmic techniques that allow us to create models that learn patterns from data.

- **AI Safety**

This refers to the techniques used to avoid or mitigate the potential catastrophic harm caused by AI systems to humans. This means that AI safety also refers to

the methodologies that allow us to deploy AI systems in safety-critical systems in compliance with safety standards.

- **Functional Safety (FuSa)**

Functional Safety is defined as the “part of the overall safety of a system that assures the freedom from unacceptable risk” [120] [4]. The aim of FuSa is to ensure that a system is free from unacceptable risk.

2 Overview

This Chapter provides an overview of the problem, its motivation, and a general overview.

2.1 Literature Classification

In this section, we explore some classifications related to AI safety-critical systems. This is useful for providing a general overview of the context of this work.

2.1.1 Types of AI

In the literature, there are many types of AI (TAI). In this section, we provide a brief recap of them, as they may be useful in the following sections.

- **Connectionists**

These are models where training is performed using an optimization technique, most commonly gradient descent. Examples include Neural Networks (e.g., CNNs, Transformers).

- **Bayesian**

These are models where results are obtained in a probabilistic manner.

- **Symbolists**

These use logic to perform reasoning (e.g., decision trees and Prolog). Given that inference must validate all parts of the logic (i.e., each component of a Boolean expression) learned during the training phase, these models are useful if we want explainable models (e.g., in the **prediction of faults** in real-time scenarios).

- **Analogizers**

These are usually classification models based on a similarity score (e.g., k-NN, SVM).

2.1.2 Automation Levels

When an AI-based system interacts with a human, the interaction can be done with different magnitudes. The EASA (European Union Aviation Safety Agency) defines three levels of automation and assistance [2]:

1. EASA Level 1

This level refers to Collaborative Robots, which work side by side with humans without having total independence.

2. EASA Level 2

The automatic systems operate in a closed environment. In this case, the system executes in a predefined environment where requirements are known during the design phase.

3. EASA Level 3

The autonomous systems operate in an open environment. ISO 22989 defines this as the highest level of automation [4], in which the system can modify its behavior and goals without human control.

2.1.3 Safety Standards

Moving from traditional methods of guaranteeing safety to using AI to achieve it requires defining a new class of standards. The most common problem is related to the probability of failure: the most critical Safety Integrity Levels (SIL, ASIL, or DAL) require a very low probability of failure (e.g., 10^{-9}), whereas excellent accuracy for an AI model is often 99%. Thus, common AI models do not comply with these requirements.

- **Traditional safety standards**

Their objective is to mitigate risks using programmable functions, known as Safety Instrumented Systems (SIS), which are embedded in the system. The reliability of a SIS is defined by different measures depending on the domain: these include the Safety Integrity Level (SIL) for industry (IEC 61508), the Automotive Safety Integrity Level (ASIL) for the automotive sector (ISO 26262), and the Design Assurance Level (DAL) for aerospace (DO-178C).

- **AI Safety Standards**

For heteronomous systems (e.g., Advanced Driver Assistance Systems (ADAS)), failures can occur even in the absence of hardware or software faults. This means that these new algorithms must comply with the Safety of the Intended Functionality (SOTIF) standard (ISO 21448), bringing them into alignment with classical FuSa.

2.1.4 Points of Application

In this section, we recap the different types of AI applications. There are three categories that describe *when* and *how* an AI model can be used.

- **Product**

A safety-critical system relies on one or more safety functions based on an AI model. This model is offline: training is performed before the model is embedded in the system.

- **Runtime**

This is similar to the previous category, but the model can update itself during

execution.

- **Process**

The system itself does not include an AI model, but it is designed and developed using AI models (e.g., generative AI to generate code or run tests).

Based on this classification, ISO 5469 defines three classes for when AI models can be included in a safety-critical system (i.e., product, runtime, and process) while complying with the standards defined in Section 2.1.3.

- **Class I**

These systems are based on traditional AI models, such as those referred to as *symbolists* in Section 2.1.1. Since they are totally deterministic and formal verification can be performed, these models comply with classic FuSa.

- **Class II**

The system contains an AI model that may not be compliant with FuSa on its own. Compliance is achieved through a runtime checker, often known as a *safety bag*; if the model fails, the checker detects the failure and makes a safe choice.

- **Class III**

These are systems that cannot incorporate an AI model internally. In this case, the solution is usually to include the model, but engineers do not take responsibility for possible failures (e.g., ADAS in vehicles).

2.2 AI-Based Safety-Critical Systems

Although integrating AI into a safety-critical system can be very challenging, there are many techniques that can be used and have already been tested. For example, referring to the classification in Section 2.1.4, we find that *Class I* and *Class II* include systems that are already on the market (e.g., SIL4 railway interlockings).

Usually, these systems are composed of the elements shown in Figure 2.1: the entire system consists of an AI system, which is defined by an execution platform and the AI item.

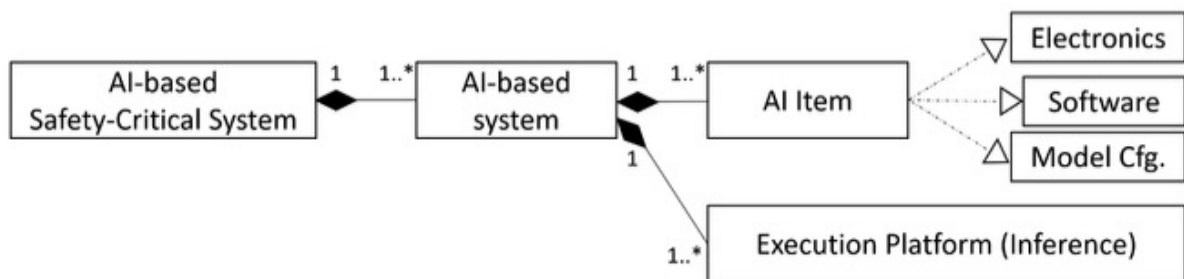


Figure 2.1: AI-Based safety-critical system components [4].

2.2.1 Product

As mentioned in Section 2.1.4, the product class describes a safety-critical system that relies for its SIS on one or more AI-based functions trained offline. These functions can be “simple”, as in the case of a symbolist AI model, thus belonging to *Class I*, or they can be very complex and belong to *Class II*. In the first case, since the model is deterministic, we can use formal verification to ensure the system’s compliance with FuSa. In the second case, the standard industry technique is to use a runtime checker that makes a safe choice for the next action if the model operates outside its requirements or produces an unsafe result.

Testing

The main problem with these AI-based safety-critical systems is the safety assurance we can assign to them. This derives from the stochastic nature of AI models, which implies that they can have *uncertainty-induced failures*; to reduce this uncertainty, we must investigate unknown factors. Therefore, when describing safety functions in this scenario, we speak about the intended functionality: their design is a set of high-level objectives rather than a set of precise and well-defined instructions. This can be mitigated by applying formal verification methods, but even in this case, the black-box nature of the models suggests that these methods cannot be the only ones. Another challenge is testing the AI model itself, which is a widely explored area (e.g., test generation using an NLP model).

Neural Networks

Although symbolist models provide understandability and explainability of their decisions, the most powerful models are Neural Networks (NNs).

When dealing with these kinds of models, we must consider several detailed issues:

- Supervised learning relies on data. When managing data (i.e., preprocessing and dividing data into training and test sets), we must do so with high precision.
- This is an area where *changing nothing changes everything* [4]. This means that changing activation functions and hyperparameters can lead to very different behaviors.
- NN training is iterative; this must be included in the V-Model during the specification and testing phases.
- AI frameworks (e.g., PyTorch) are general-purpose and user-friendly. However, there are some APIs that also provide a set of safety requirements.

2.2.2 Runtime

As discussed in Section 2.1.4, a runtime safety-critical system is able to manage its AI module during execution. These are notable systems because they allow the SIS to evolve with the context: a case study reported a gas turbine where the safety function evolves as the engine degrades.

This kind of solution bases its dependability and standard compliance on five different techniques:

- **Safety Bag**

This is the same technique mentioned above: a runtime checker monitors the AI model's results, and if it detects anomalous behavior, it produces a safe output.

- **Safe Adaptation**

Both the AI model and the (continuous) training algorithm conform to safety standards.

- **Limited Adaptation**

The (continuous) training is limited within a safe range: this means that even if the weights of the model change, they cannot do so by a large magnitude.

- **Limited Actuation**

In this case, the adaptation is performed with hard constraints on the resulting model's output: for example, the new model cannot produce an output whose magnitude exceeds a certain threshold.

- **Library-Based Offline**

We train a set of AI models, each of which conforms to safety standards. During inference, the system chooses among the available models in a non-linear way (i.e., with if/else).

2.2.3 Process

These are offline solutions that support engineers in designing safety-critical systems, both for the overall system and the AI component that implements the safety function.

In traditional safety engineering, AI models can be useful for validating and assessing the V-Model. During specification, we can use an NLP model (e.g., Transformer-based) to discover hazards and issues in the process; during design and implementation, models can be useful for generating code; and during the testing phase, they are useful for generating test cases.

If we look at the use of AI to construct other models that will implement safety functions, there are different solutions: in AutoML, we use reinforcement learning to find the best model hyperparameters, and we can implement models that search exhaustively for all combinations to test our system.

3 Runtime Monitoring

As described in Section 2.2.2, there is a notable class of systems, referred to as runtime, which are able to handle the complexity and non-compliance of their AI models with safety standards. These systems rely on different solutions to manage their complex algorithms, one of which is known as runtime monitoring. In this scenario, one of the main challenges is to obtain certification when a module is present within the system that does not have deterministic behavior, is very complex to handle in a formal way, and can produce undetectable faults in the system.

One of the most practical solutions to this problem is *runtime assurance* (RTA): we do not try to make the complex module compliant, but instead we want to predict its faulty behavior and, in this case, switch to a safe module. This approach has high modularity: once we have obtained certification for the whole system, we can change the AI module without problems, because if it fails, the safe function enters into action. Another interesting point is that this solution can be applied at any system level (e.g., flight controller and actuation system) [3].

3.1 Architecture

The idea we described in the introduction of this chapter is formalized in the *ASTM F3269-17* standard (the latest version is *F3269-21*, but it must be purchased). In 2017, the Advancing Standards Transforming Market (ASTM) published a guideline to implement this behavior on unmanned aircraft systems. Figure 3.1 shows a diagram of the proposed architecture.

Let's see some terms that are useful to know:

- **Data**

Obviously, data is the information that will be processed by the system. These data can be both assured and unassured: in the first case, we are referring to data that can be used directly by the system; in the second one, we are referring to data that must be validated through the **Input Manager**.

- **Complex Function**

This is the module that cannot be certified, usually an AI module. This AI component is composed of data pre-processing, inference, and post-processing; furthermore, this component can be implemented with some degree of redundancy, with a voter at the end [1].

- **Recovery Function**

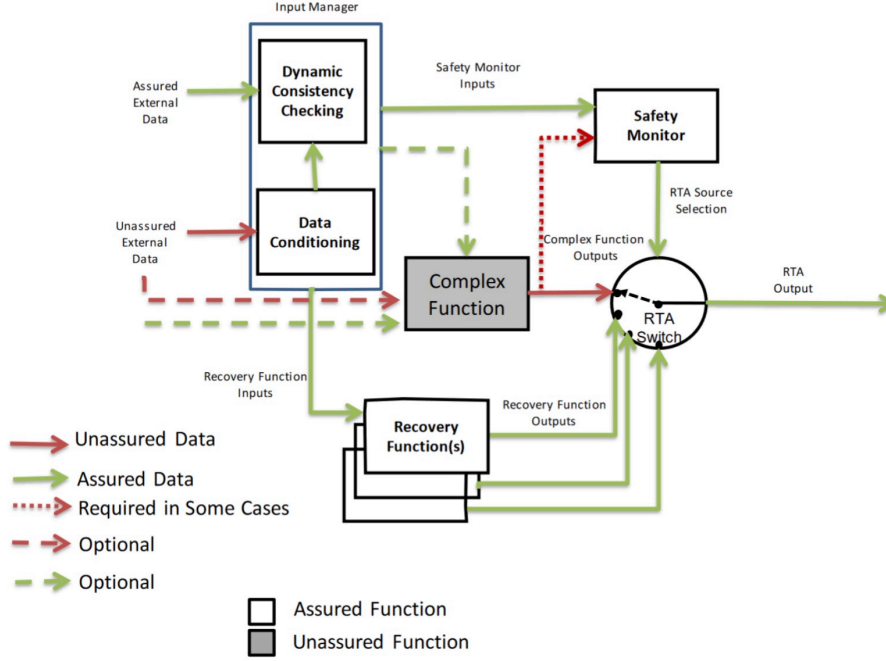


Figure 3.1: RTA architecture Proposed in ASTM F3269-17 [3].

It is an assured function that generates a bounded output to maintain the system in a safe state if the Complex Function fails.

- **Safety Monitor**

The assured function that is responsible for detecting faulty behavior and sending commands to the Switch. This module can observe both the RTA system and the Complex Function; this choice depends on the use case. This monitor can also perform the analysis on the AI module in a hierarchical way (see Figure 3.2): the low-level monitor checks for evident errors (e.g., out-of-distribution values) and the high-level monitor checks for more semantic errors [1].

- **Switch**

An assured function that receives the switching command from the Safety Monitor and switches to the Recovery Function.

3.2 Coverage

The coverage concept is fundamental to make the RTA system work. We start from the operational scope of the Complex Function, i.e., from the boundary of the functionality that the Complex Function implements. This scope is analyzed from the viewpoint of all combinations of Safety Monitors and Recovery Functions (see Figure 3.3): for each operating point (i.e., functionality) of the scope, there must be a monitor capable of detecting the Complex Function's failure and a Recovery Function that maintains the system in a safe state.

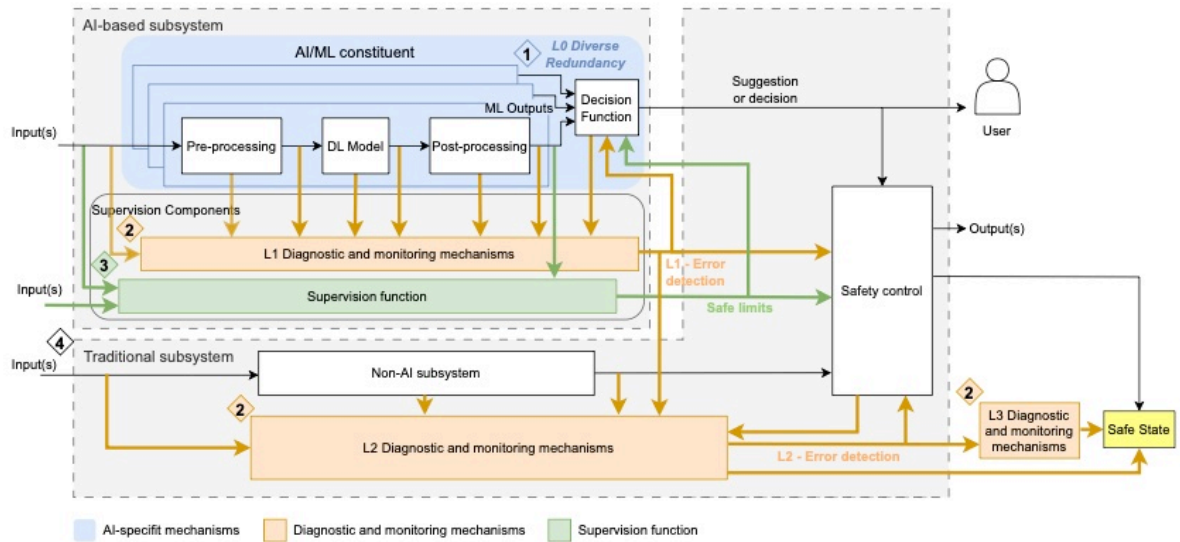


Figure 3.2: RTA architecture with Hierarchical Monitor [2].

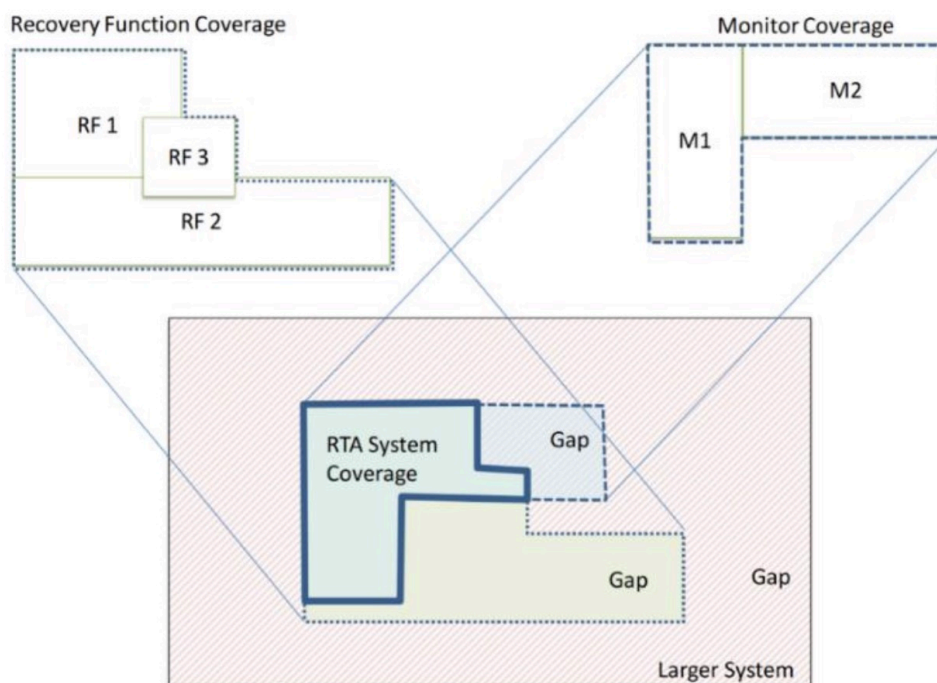


Figure 3.3: Gap in the Monitor and the Recovery Function [3].

This concept of coverage leads us to have multiple monitors, where each of them has its own Recovery Function. The choice of which monitor and its relative Recovery Function is performed by a voter, resulting in an architecture similar to the one shown in Figure 3.4.

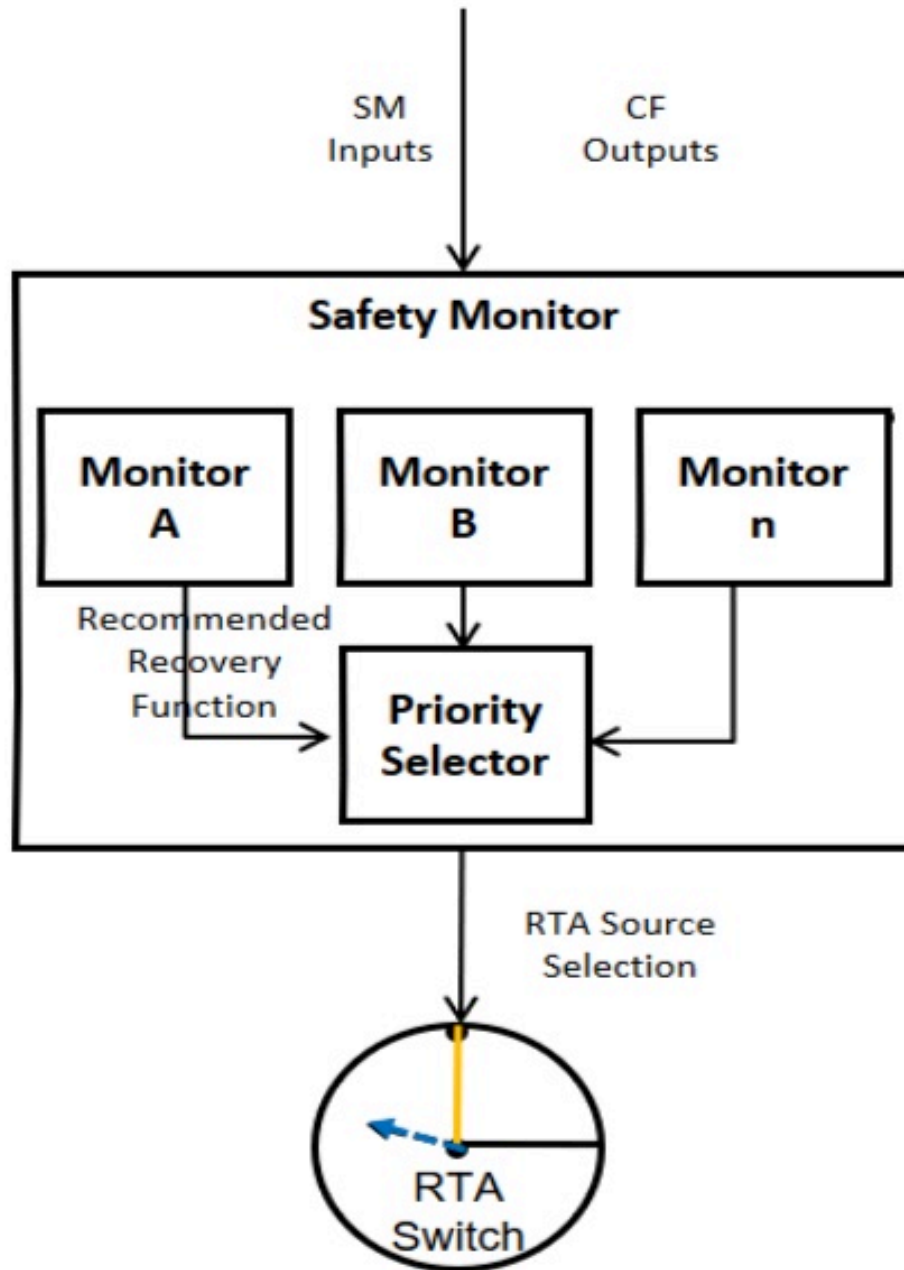


Figure 3.4: Voter architecture with multiple Monitors [3].

4 Implementation

In Chapter 3, we saw the theory behind Runtime Assurance. In this chapter, we will implement something related to this that follows the idea of runtime monitoring.

In this chapter, we present all the information needed to understand and go into detail about the activity diagram in Figure 4.1 and the sequence diagram in Figure 4.2.

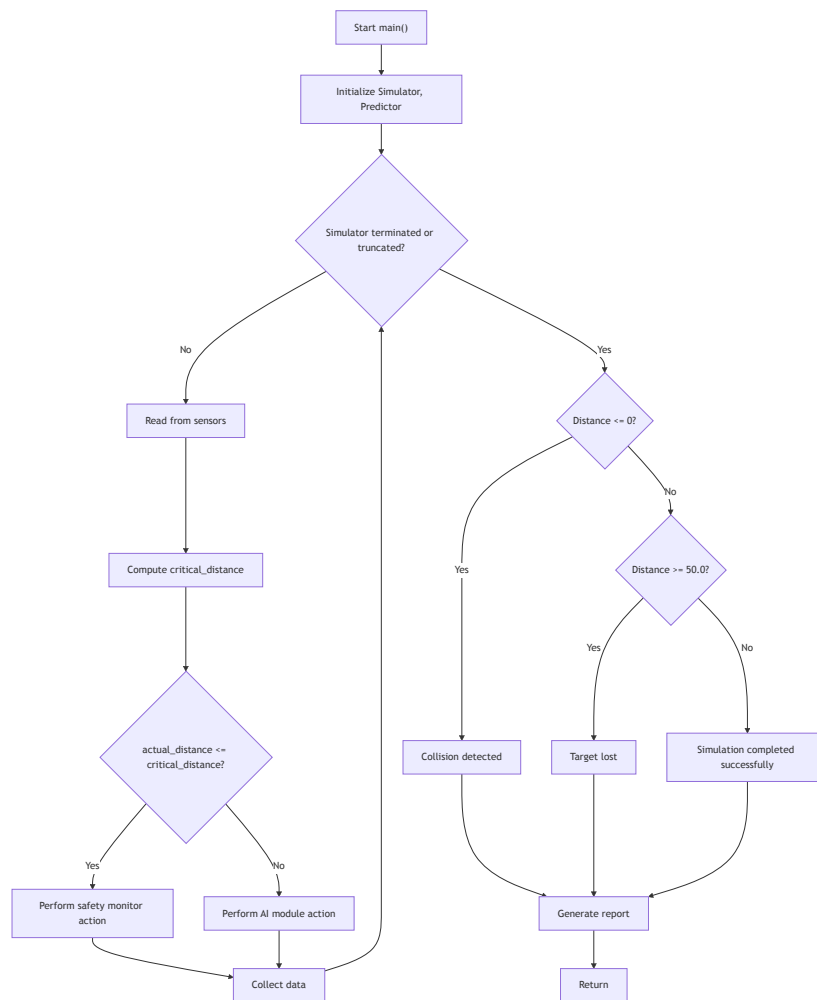


Figure 4.1: Activity Diagram

4.1 Scenario

The objective is to implement a safety-critical Advanced Driver Assistance System (ADAS) using the RTA pattern. The idea is that the Ego is managed by the AI Module until it fails; if and when the AI fails, the safety module takes control and applies a safe behavior implemented in the recovery function.

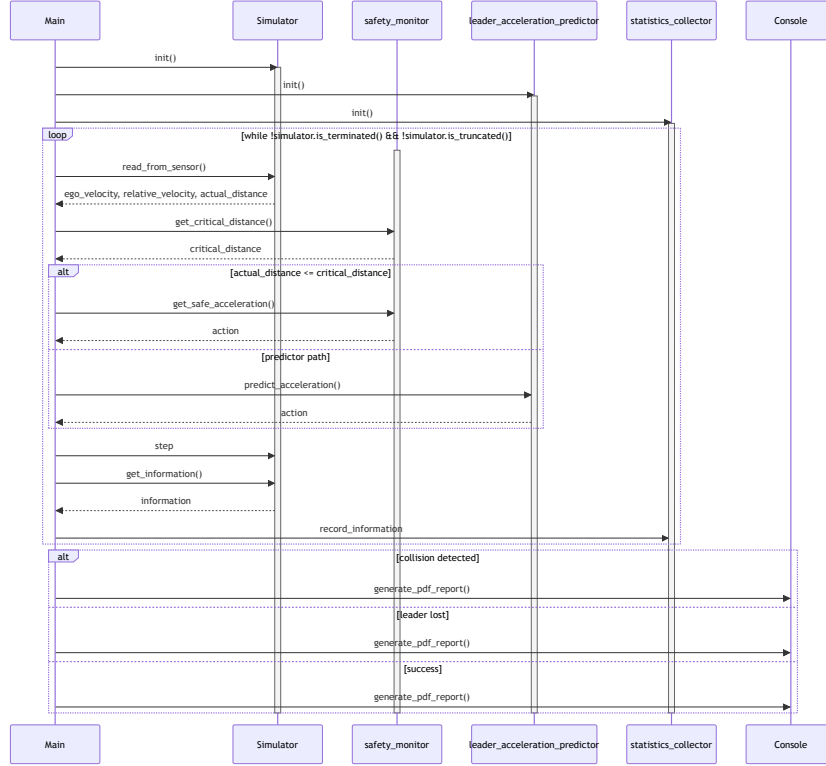


Figure 4.2: Sequence diagram.

4.1.1 Components

This scenario is based on two components:

- **Leader Car (L)**

It's the independent vehicle that goes ahead. It is governed by an acceleration $a_L(t)$ that is unknown from the point of view of our system (i.e., the Ego Car).

- **Ego Car (E)**

It's our vehicle, over which we have control, and represents our safety-critical system. In the same way as L , we have control over E through the acceleration function $a_E(t)$.

4.1.2 Acceleration Function

The idea behind the acceleration function $a_L(t)$ is the following: we want a continuous and differentiable function; if the function is deterministic episode by episode, the agent only learns the current pattern. So we have to define a function in order to have a deterministic acceleration during a single episode, but non-deterministic episode by episode.

The function can thus be described as

$$a_L(t) = \begin{cases} \text{clip}(A \sin(\omega t + \phi), -8.0, 3.0) & \text{if } t < t_{brake} \\ a_{max_brake} & \text{if } t \geq t_{brake} \text{ and } v_L(t) > 0 \\ 0 & \text{otherwise,} \end{cases} \quad (4.1)$$

where:

- t_{brake}

With a given probability during the episode, the Leader car brakes with the maximum force.

$$t_{brake} = \begin{cases} \tau \sim \mathcal{U}(5, 20) & \text{with } p = 0.3 \\ \infty & \text{with } p = 0.7 \end{cases} \quad (4.2)$$

- A

It represents the maximum magnitude of the acceleration. If it is greater than the maximum power of the engine, the result is a constant a_L . It is defined as $A \sim \mathcal{U}(1.0, 5.0) \text{ m/s}^2$.

- ω

It is the frequency of the sinusoidal function. It is defined as the angular frequency $\omega \sim \mathcal{U}(0.1, 0.4) \text{ rad/s}$.

- ϕ

It is the translation point and represents the initial value of a_L in the simulation. It is defined as $\phi \sim \mathcal{U}(0, 2\pi)$.

4.2 Simulator

The data to train the neural network and for inference are provided by a simulator. The maximum duration of a single episode is 30s.

4.2.1 Kinematics Rules

The simulation evolves following second-order kinematics rules. These rules are defined in a discrete way with a time-tick of $\Delta t = 0.1 \text{ s}$, and so we have $t + 1 = t + \Delta t$.

$$v_L(t + 1) = \max(0, v_L(t) + a_L(t) \cdot \Delta t), \quad (4.3)$$

$$x_L(t + 1) = x_L(t) + v_L(t) \cdot \Delta t + \frac{1}{2} a_L(t) \cdot \Delta t^2, \quad (4.4)$$

$$v_E(t + 1) = \max(0, v_E(t) + a_E(t) \cdot \Delta t), \quad (4.5)$$

$$x_E(t + 1) = x_E(t) + v_E(t) \cdot \Delta t + \frac{1}{2} a_E(t) \cdot \Delta t^2, \quad (4.6)$$

where: the max function means that the two cars can only move forward; $x(t)$ denotes position; $v(t)$ denotes velocity; $a(t)$ denotes the acceleration and is bounded by the vehicle constraints, in this case defined as $-8.0 \text{ m/s}^2 \leq a(t) \leq 3.0 \text{ m/s}^2$ for both vehicles.

4.2.2 State

The observable system states accessible to the controllers at any step t are:

- $d(t) = x_L(t) - x_E(t)$, $v_{rel}(t) = v_L(t) - v_E(t)$
These are the distance between the two vehicles and their relative velocity. This information is provided through radar sensors (e.g., LiDAR).
- $v_E(t)$
This is the Ego absolute velocity. It is accessible through the Ego car's sensors.

4.2.3 Initial Conditions

At the beginning of each episode, at $t = 0$, the initial state of the environment is sampled from uniform distributions to ensure generalization:

- $x_E(0) = 0 \text{ m}$ (The Ego car starts at the origin).
- $d(0) \sim \mathcal{U}(5.0, 20.0) \text{ m}$.
- $v_L(0) \sim \mathcal{U}(10.0, 20.0) \text{ m/s}$ (Initial velocity of the Leader car).
- $v_E(0) \sim \mathcal{U}(10.0, 20.0) \text{ m/s}$ (Initial velocity of the Ego car).
- $a_E(-1) = 0.0 \text{ m/s}^2$ (Assumed previous acceleration for the first-step comfort reward).

4.3 Architecture

Referring to Section 2.1.4, the goal is to implement a *Product* system of *Class II*: we have a complex component, the AI network that predicts $a_E(t)$, which is non-deterministic and untrusted but is guaranteed by a monitor module.

4.3.1 AI Module

The AI module, trained with the Reinforcement Learning (RL) paradigm, takes as in-

put a three-dimensional tensor $S_t = \begin{bmatrix} v_E(t) \\ d(t) \\ v_{rel}(t) \end{bmatrix} \in \mathbb{R}^3$; its task is to predict the acceleration $a_E(t) \in [-8.0, 3.0] \subset \mathbb{R}$ of the Ego vehicle.

Since we have to predict a function, the neural network is a Multi-Layer Perceptron (MLP): the activation function is a ReLU; the input layer has 3 neurons (i.e., the input

size); the output has only 1 neuron (i.e., the scalar value of the acceleration); the hidden layers are composed of [16, 32, 32, 16] neurons.

4.3.2 Safety Monitor

This is the deterministic and assured module that takes control of the Ego car in case of AI failure. It computes the minimum distance that should be maintained between the two cars in the situation where the Leader brakes with maximum power. This safety distance can be defined as

$$d_{critical} = \max \left(0, v_E(t) \cdot \Delta t + \frac{v_E(t)^2 - v_L(t)^2}{2 \cdot |a_{max_brake}|} \right), \quad (4.7)$$

where $v_E(t) \cdot \Delta t$ is the reaction time of the system, which is necessary because the braking phase does not start immediately but only after Δt ticks; $v_L(t) = v_{rel}(t) + v_E(t)$; and $a_{max_brake} = -8.0$.

The safety monitor has to take control of the system if $d(t) \leq d_{critical}$.

4.3.3 Recovery Function

The recovery function implements a fail-safe behavior: the E vehicle will stop with the maximum braking power (i.e., $a = -8.0 \text{ m/s}^2$).

4.4 Implementation Details

In this section, we will go into detail about the implementation choices.

4.4.1 Algorithm

The environment is provided by the Gymnasium API. The training is performed using the PPO algorithm from the Stable-Baselines3 library, which is more stable than REINFORCE. The hyperparameters of this algorithm are the following: $\gamma = 0.99$; the number of samples for each partial update is $batch_size = 64$; $n_steps = 1024$ is the number of steps (about 3/4 episodes) after which the update is performed; and $clip_range = 0.2$ prevents the network from changing by more than 20%.

4.4.2 Environment

The simulator is written in C++.

For the neural network training, the functions that the environment calls are the same as those used during inference, and this is guaranteed by pybind11, which allows us to call C++ code directly from Python.

For inference, the neural network will be embedded in the system using the ONNX

API.

4.4.3 Reward Function

The reward function is usually the most critical part in RL. In this case, we want to provide both safe and comfortable behavior, so we can describe the reward function as $R_t = R_{distance} + R_{comfort} + R_{penalty}$, where:

- $R_{distance} = -\alpha|d(t) - d_{target}|$, where $d_{target} = 10$ m, with $\alpha = 1$.
- $R_{comfort} = -\beta|a_E(t-1) - a_E(t)|$, where we want to penalize sudden braking, with $\beta = 0.9$.
- $R_{penalty}$ represents the agent entering the critical zone, i.e., if $d(t) \leq d_{critical}$. It is defined as $R_{penalty} = \begin{cases} -20 & \text{if } d(t) \leq d_{critical} \\ 0 & \text{otherwise} \end{cases}$.

We immediately return a reward of $R_{terminal} = -1000$ if $d(t) \leq 0$ or if $d(t) > d_{max} = 50$ m. In both cases, the episode is immediately terminated.

4.4.4 Episode Termination

The episode doesn't terminate when $d(t) \leq d_{critical}$. Instead, the simulation continues while the agent receives a negative reward. The episode terminates immediately if:

- Collision: $d(t) \leq 0$ (Terminal penalty: $R_{terminal} = -1000$)
- Leader Lost: $d(t) > d_{max} = 50$ m (Terminal penalty: $R_{terminal} = -1000$).
- Timeout: The episode reaches its maximum duration of 30 s.

4.5 Results

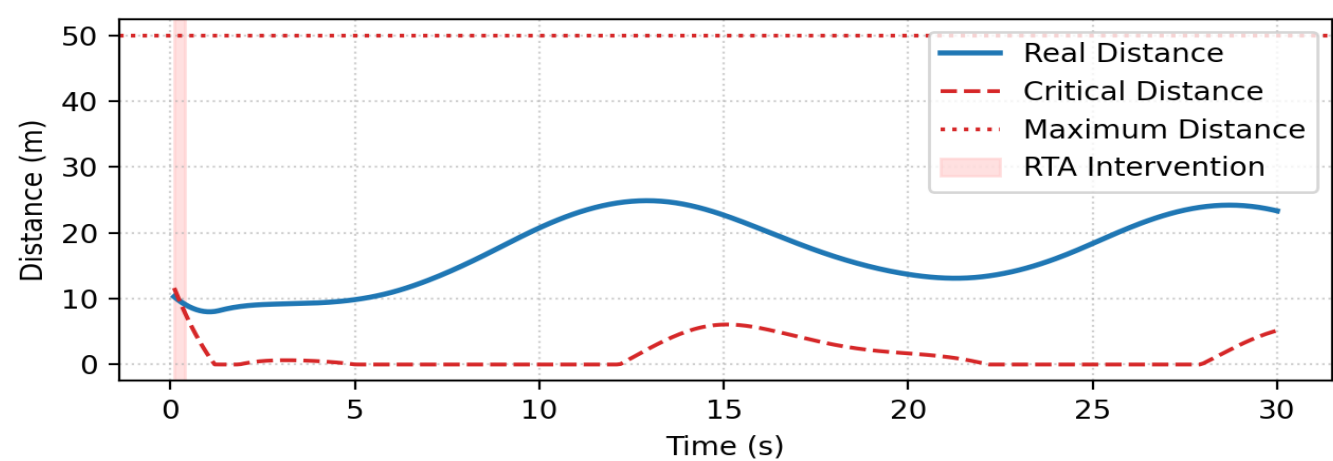
In this Section, we append some tests with corresponding results:

AI Safety Critical Runtime Monitoring

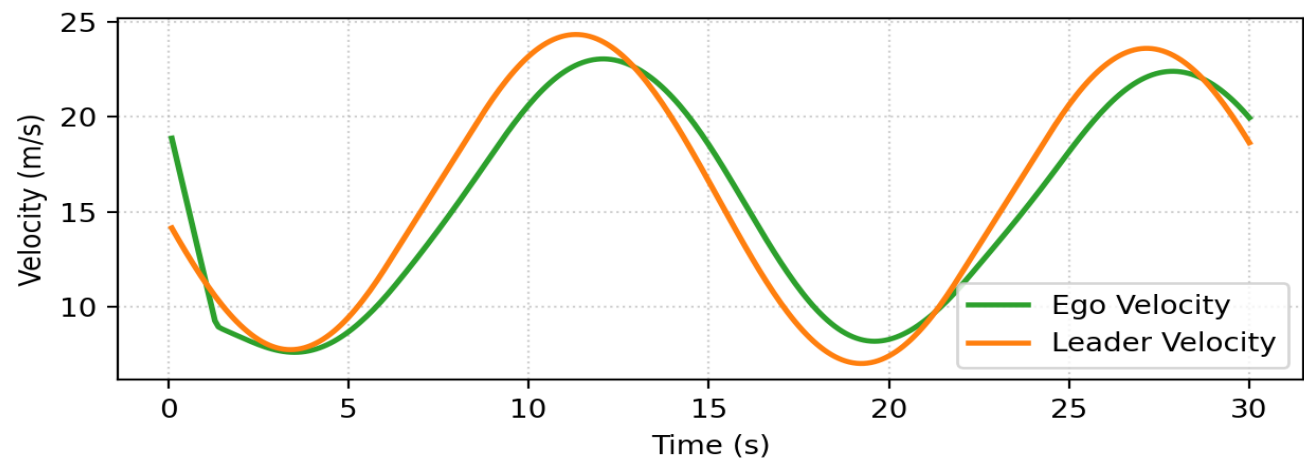
Simulation completed sucessfully!

Metric	Value
Total Time	30.00 s (300 ticks)
AI Decisions	297 (99.0%)
RTA Interventions	3 (1.0%)
Minimum Distance	8.01 m
Average Distance	17.03 m (Target: 10m)
Maximum Distance	24.90 m
Maximum Ego Velocity	23.04 m/s (83.0 km/h)
Average Ego Velocity	15.23 m/s (54.8 km/h)

Distance Chart



Velocity Chart

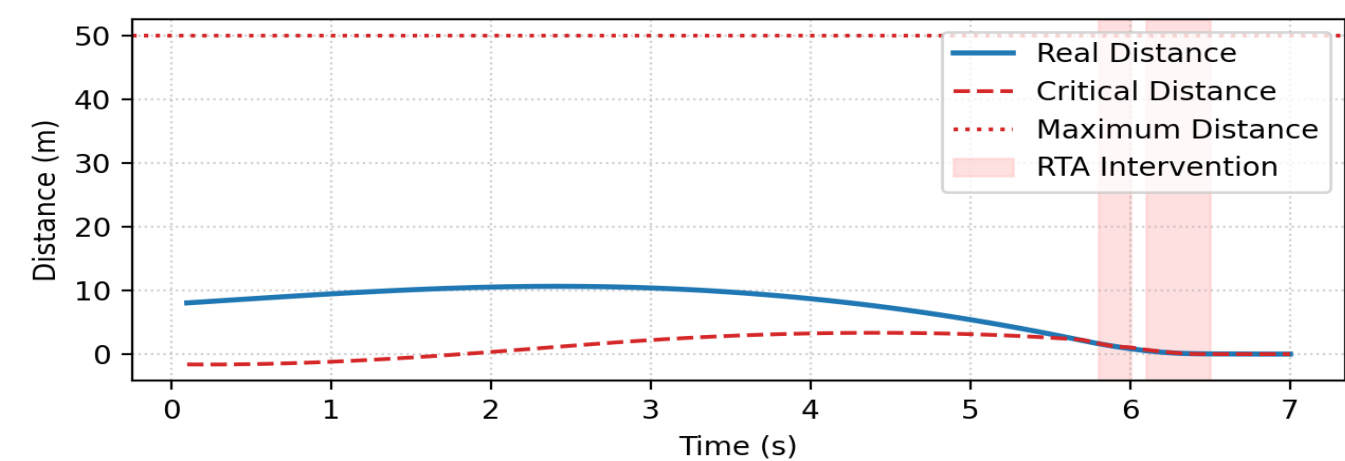


AI Safety Critical Runtime Monitoring

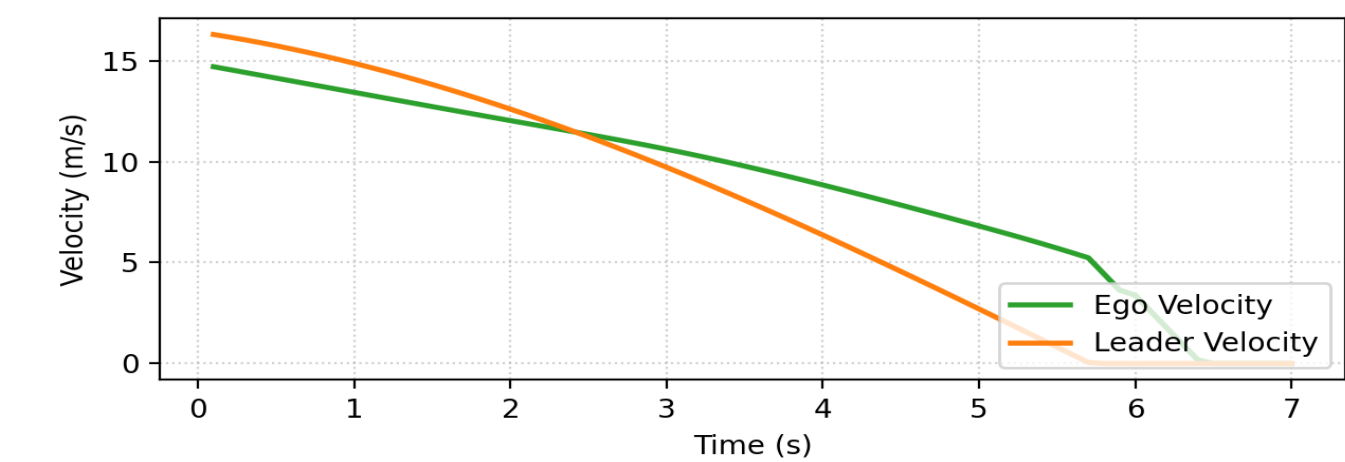
Collision detected!

Metric	Value
Total Time	7.00 s (70 ticks)
AI Decisions	64 (91.4%)
RTA Interventions	6 (8.6%)
Minimum Distance	0.00 m
Average Distance	7.00 m (Target: 10m)
Maximum Distance	10.63 m
Maximum Ego Velocity	14.70 m/s (52.9 km/h)
Average Ego Velocity	8.75 m/s (31.5 km/h)

Distance Chart



Velocity Chart

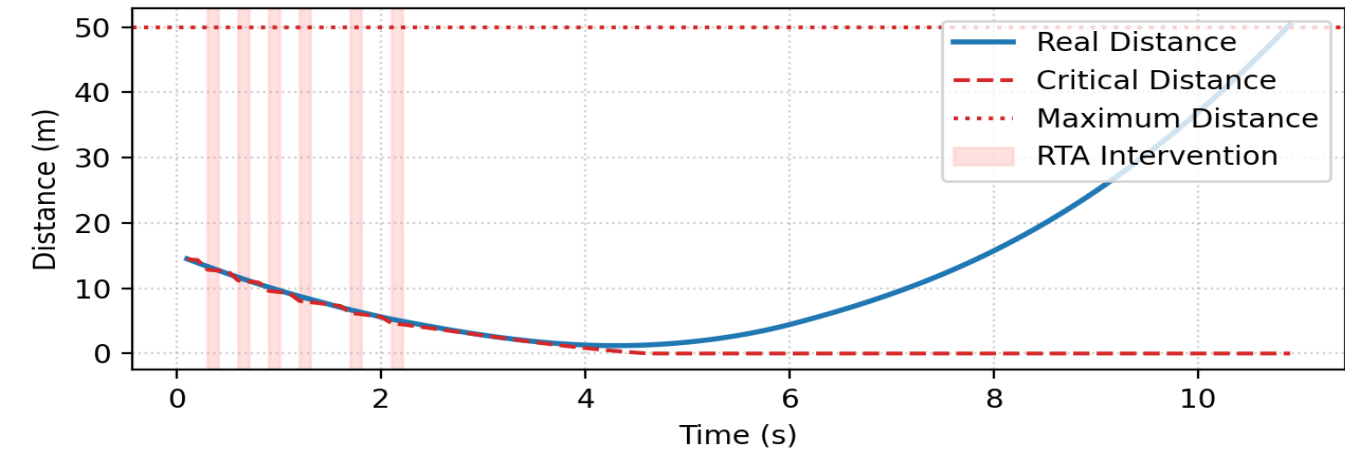


AI Safety Critical Runtime Monitoring

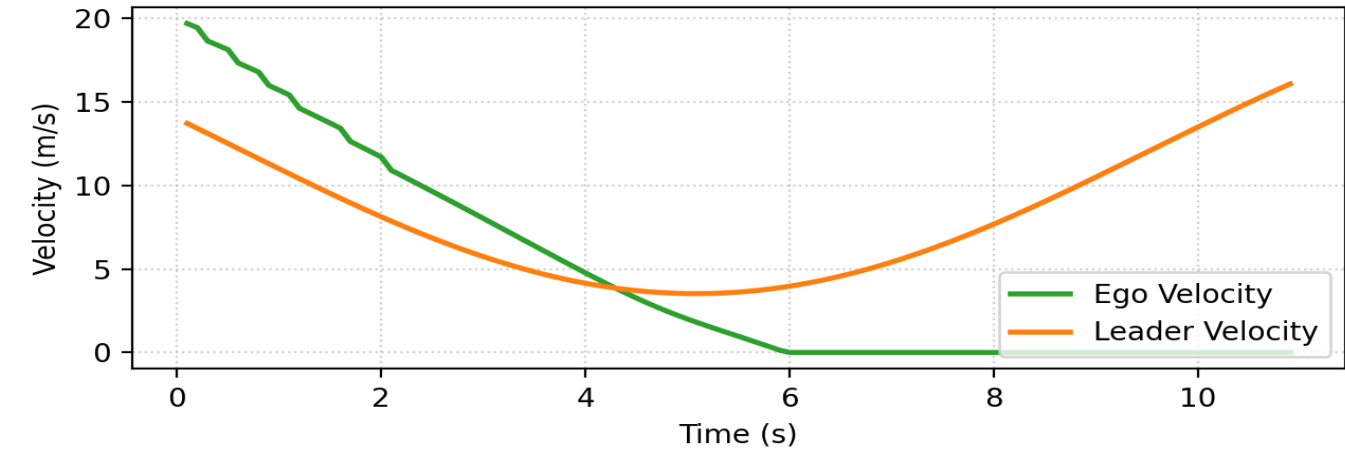
Leader vehicle lost (tracking failed)!

Metric	Value
Total Time	10.90 s (109 ticks)
AI Decisions	103 (94.5%)
RTA Interventions	6 (5.5%)
Minimum Distance	1.20 m
Average Distance	12.91 m (Target: 10m)
Maximum Distance	50.40 m
Maximum Ego Velocity	19.69 m/s (70.9 km/h)
Average Ego Velocity	4.67 m/s (16.8 km/h)

Distance Chart



Velocity Chart



Bibliography

- [1] Jaume Abella et al. "SAFEXPLAIN: a Complete Approach Towards Trustworthy AI-Based Safety-Critical Systems". In: *2025 28th Euromicro Conference on Digital System Design (DSD)*. IEEE. 2025, pp. 324–331.
- [2] Irune Agirre et al. "Functional Safety Architectural Patterns for AI-Based Critical Systems". In: *ACM Transactions on Cyber-Physical Systems* (2025).
- [3] Pranav Nagarajan et al. "ASTM F3269-an industry standard on run time assurance for aircraft systems". In: *AIAA Scitech 2021 Forum*. 2021, p. 0525.
- [4] Jon Perez-Cerrolaza et al. "Artificial intelligence for safety-critical systems in industrial and transportation domains: A survey". In: *ACM Computing Surveys* 56.7 (2024), pp. 1–40.